Cipher Maps: Total Functions as Trapdoor Approximations

Alexander Towell
lex@metafunctor.com

March 2026

Abstract

Outsourcing function evaluation to an untrusted machine typically requires oblivious RAM (hiding access patterns), fully homomorphic encryption (exact computation on ciphertexts), or garbled circuits (encrypted lookup tables, single-use). Each carries substantial cost. We propose the *cipher map*: a total function on bit strings whose privacy properties emerge from a one-way trapdoor (a cryptographic hash with a secret seed) combined with a frequency-equalized output distribution. Cipher maps fit the quantitative information flow tradition [25, 2]: rather than achieving negligible leakage in a security parameter, they bound observable leakage by measurable parameters δ (representation uniformity), ε (noise-decode probability), and η (correctness), with the entropy ratio serving as the security measure (formal framework in companion work [29]). The construction reduces to a single design choice: an *acceptance predicate* that partitions hash space among output values. Shannon-optimal allocation of this partition simultaneously minimizes space (achieving $-\log_2 \varepsilon + H(Y)$ bits per element, the information-theoretic lower bound), maximizes output indistinguishability (noise and real outputs share the same frequency profile), and enables predictable error composition ($\eta_{\text{total}} \leq 1 - \prod(1 - \eta_i)$). We formalize the framework through four measurable properties, prove the composition and space-optimality theorems, and illustrate the construction on arbitrary maps, set membership, and encrypted search.

1 Introduction

Consider a setting in which a trusted party wishes to outsource computation to an untrusted machine. The trusted party holds a function $f : X \to Y$—a lookup table, a set membership predicate, an index—and wants the untrusted machine to evaluate f on encoded inputs without learning f, X, or Y.

The central object of this paper is the *cipher map*: a total function $\hat{f} : \{0,1\}^n \to \{0,1\}^n$ that the untrusted machine evaluates blindly. The key insight is that $\hat{f}$ is total—every n-bit input produces an n-bit output, with no concept of "in-domain" or "out-of-domain." The notions of domain, correctness, false positive, and false negative exist only on the trusted machine, which holds the encoding and decoding functions (the trapdoor). The untrusted machine sees only opaque bit strings flowing through opaque total functions.

This paper develops the theory of cipher maps in a self-contained way. We define four properties that characterize a cipher map (§4), formalize the trusted/untrusted machine model (§5), develop the batch construction unified under the acceptance predicate framework (§6), prove the composition theorem governing error accumulation (§7), and discuss the online alternative via trapdoor Boolean algebra (§9.3).

Contribution: framework, not new construction. We do not propose a new construction with novel space, time, or correctness guarantees; rather, we identify a *framework* (the cipher map abstraction with four measurable properties and the acceptance predicate apparatus) that unifies several existing approximate-membership and frequency-hiding constructions under a single formalism with measurable parameters $(\eta, \varepsilon, \delta, \mu)$. Bloom filters become cipher maps with $K(x) = 1$, exact correctness ($\eta = 0$), and a degenerate acceptance partition; prefix-free coding instantiates the entropy cipher map; Shannon-optimal partition shaping unifies space optimality with frequency hiding (§6.2, Figure 1). The contributions are the framework itself plus the formal results that follow from it: an information-theoretic lower bound matched by the construction (Theorems 6.1, 6.2), a composition theorem with error bounds (Theorem 7.2), a confidentiality bound tying representation uniformity to a quantitative leakage measure (Proposition 5.1), and a unification of three prior threads (approximate data structures, frequency-hiding encryption, encrypted search) under one abstraction.

Bernoulli error model. The error framework underlying cipher maps is the Bernoulli model [27]. Two axioms (element-wise independence and conditional independence of block error rates) reduce the error model from exponential complexity to two parameters per element. This tractability is what makes the composition theorem (Property 4) possible: errors combine predictably because they are independent across elements. The Bernoulli model is developed independently; we reference it here only for the error accounting that cipher maps inherit.

Positioning: parameterized leakage in the QIF tradition. Cipher maps belong to the *quantitative information flow* tradition [25, 2] and the broader parameterized-leakage program initiated by entropic security [12]. The defining commitment is that security is *measurable rather than negligible*: an untrusted evaluator's residual uncertainty about the latent function is reported as a continuous quantity (the *entropy ratio* $e = H(Q)/n$ where Q is the cipher value distribution observed by the untrusted evaluator and n is the cipher value space dimension), bounded below by the cipher map's representation-uniformity parameter δ via the Fannes–Audenaert continuity inequality (formal framework in companion work [29]). This puts cipher maps in a different design space from oblivious RAM [17] (hides access patterns at polylog overhead), fully homomorphic encryption [16] (preserves exact algebraic structure on ciphertexts), and simulation-based searchable encryption [11] (achieves indistinguishability up to a leakage profile). The closest structural relative is the garbled circuit [32], which also encrypts a lookup table; cipher maps differ in being reusable, approximate, and parameterized. We make the contrast precise in §2.

2 Related Work

Cipher maps draw on and differ from several lines of work in data structures, cryptography, and secure computation.

Membership data structures. Bloom filters [7] are the canonical approximate membership structure: a bit array with k hash functions, trading false positives for space. Quotient filters [6] and cuckoo filters [14] improve cache locality and support deletion, respectively, but share the same basic model. Set membership (Remark 6.3) subsumes Bloom filters as a special case: a single hash function ($k = 1$), exact correctness ($\eta = 0$), and the same information-theoretic space bound of $-\log_2 \varepsilon$ bits per element. The difference is that the HashSet is a cipher map—a total function on bit strings with a trapdoor—while Bloom filters expose their output directly.

Perfect hash functions. Fredman, Komlós, and Szemerédi [15] introduced FKS perfect hashing with $O(n)$ space and $O(1)$ lookup. Belazzougui, Botelho, and Dietzfelbinger [3] achieved minimal perfect hash functions near the information-theoretic lower bound. The batch cipher map construction (§6) is a perfect hash function with a different optimization target: it maps domain elements to prefix-free codewords (not to consecutive integers) and trades correctness ($\eta > 0$) for space, achieving $(1 - \eta)(-\log_2 \varepsilon + H(Y))$ bits per element.

Property-preserving encryption. Order-preserving encryption (OPE) [1, 8] and deterministic encryption [5] enable computation on ciphertexts by preserving algebraic structure (order, equality). Naveed et al. [22] demonstrated devastating inference attacks exploiting exactly this preserved structure. Cipher maps differ fundamentally: rather than preserving structure at the cost of leakage, cipher maps hide structure behind a total function. The untrusted machine cannot distinguish domain elements from noise, and the output distribution is controlled by the representation uniformity parameter δ.

Searchable symmetric encryption. Song, Wagner, and Perrig [26] introduced practical encrypted keyword search. Curtmola et al. [11] formalized SSE with simulation-based security (IND-CKA). Cash et al. [9] extended SSE to Boolean queries with sublinear search. SSE constructions use inverted indices and game-based security definitions; cipher maps use total functions with information-theoretic parameters (η, ε, δ). The approaches address different threat models: SSE hides query results from the server (up to leakage profiles), while cipher maps hide function identity and domain membership through totality and noise closure. The volume-hiding line [20] is closer to cipher maps' approach: rather than analyzing leakage, it engineers indistinguishability between document-set sizes. Cipher maps achieve a related guarantee structurally (totality plus Property 2) at lower per-query cost.

Frequency-hiding constructions. Kerschbaum's frequency-hiding OPE [21] hides plaintext frequency by injecting random ranks on duplicate inserts; cipher maps hide it by shaping the acceptance partition so that the cipher value distribution matches the output distribution (§6.2, Figure 1). Both attack the same threat (frequency analysis on encrypted records) by different mechanisms: Kerschbaum randomizes duplicate ranks so that frequency information cannot be recovered from rank ordering, while cipher maps allocate hash-space partition widths $\alpha(y) \propto p_y$ so that frequency information cannot be recovered from output marginals. The two constructions are complementary: Kerschbaum's mechanism applies within each cipher map evaluation, the cipher map mechanism applies across many evaluations.

Leakage-abuse attacks. A line of work demonstrates that the leakage permitted by SSE and PPE security definitions is exploitable in practice. Islam, Kuzu, and Kantarcioglu [18] showed that access-pattern leakage on SSE indices enables query recovery via background-knowledge attacks. Naveed, Kamara, and Wright [22] demonstrated devastating frequency- and inference-based attacks on PPE-encrypted databases (OPE, deterministic encryption), recovering plaintext columns from auxiliary data. Cash, Grubbs, Perry, and Ristenpart [10] showed that even SSE schemes with leakage profiles formally analyzed under simulation-based definitions remain vulnerable to query- and document-recovery attacks under realistic adversary knowledge. Cipher maps target the upstream causes of these attacks: rather than permitting leakage and analyzing it, the four properties bound what U can observe. Totality and Shannon-optimal acceptance allocation together flatten

the visible frequency profile (Property 2, δ-bounded), so frequency-based attacks face uniform output; totality eliminates the "in-domain vs. out-of-domain" distinction exploited by access-pattern attacks. These structural defenses do not make cipher maps immune to all leakage-abuse vectors: composition leaks correlation structure (§8.2), and a bounded query budget combined with auxiliary data still gives the adversary statistical advantage.

Honey encryption and "every input decodes." Honey encryption [19] produces plausible-looking plaintexts under any decryption key, defeating brute-force key search because the adversary cannot tell a wrong key's output from a real plaintext. Cipher maps share the structural property that every input produces output: there is no failure mode that signals "wrong key" or "not in domain." But the targets differ. Honey encryption protects *the encryption of a single message* against *key recovery*; cipher maps protect *the function f* against repeated function evaluation by an untrusted evaluator. Honey encryption is exact (a real key recovers the real message); cipher maps are approximate (correctness $1 - \eta$ even under the real trapdoor). The shared intuition—that ambiguity at decode time provides deniability—scales differently: cipher maps' noise-decode probability ε converts every cipher output, including outputs under the real trapdoor, into a value that is either the latent answer or a uniformly drawn alternative.

Garbled circuits and fully homomorphic encryption. Yao's garbled circuits [32] evaluate arbitrary functions on encrypted inputs via encrypted lookup tables. Gentry's fully homomorphic encryption [16] enables exact computation on ciphertexts without interaction. Both achieve exact computation on encrypted data. Cipher maps trade exactness for simplicity: a cipher map is a static lookup table (one hash evaluation per query) with tunable error η, whereas garbled circuits require per-gate encryption and are single-use, and FHE incurs substantial computational overhead from noise management in lattice operations.

Homophonic substitution. The representation uniformity property (multiple encodings per value, $K(x) \propto 1/D(x)$) is a modern instance of homophonic substitution [24], in which frequent plaintext symbols are assigned multiple ciphertext equivalents to flatten frequency distributions. The connection is direct: representation uniformity generalizes homophonic substitution from substitution ciphers to arbitrary total functions.

3 The Cipher Map Abstraction

3.1 Preliminaries

Throughout, $h : \{0,1\}^* \to \{0,1\}^n$ denotes a cryptographic hash function modeled as a *random oracle* [4]: a truly random function accessible to all parties via oracle queries. Results stated "under the random oracle model" assume h has this property. Elements of any finite type are encoded as bit strings before hashing.

ROM dependence and standard-model considerations. All four properties and every theorem in this paper depend on the random oracle model. Practical instantiation uses cryptographic hash functions (e.g., SHA-256, BLAKE3) whose pseudorandomness is conjectured but not proven. The critical ROM assumptions are: *(a)* uniform hash output on non-stored elements (Property 1, totality, and the noise-decode probability ε); *(b)* independence of hash values across distinct elements

(Property 3, correctness analysis, and the construction-time bound of Proposition 6.4); *(c)* independence across maps with distinct seeds (Property 4, composition); *(d)* independence of hash values from the seed for unstored inputs and across multiplicities $k \in \{0, \ldots, K(x) - 1\}$ (Property 2, the δ-bounded representation uniformity argument underlying Proposition 5.1). For domain-separated keyed hashes (HMAC-SHA256 with seed ℓ as the key), each of these is widely accepted in practice. A standard-model instantiation via universal hash families or pairwise-independent constructions is an open question; we expect the four-property framework to survive such a translation, but with explicit dependence on collision-resistance and key-distinguishability assumptions.

Notational convention. We write $\mathsf{C}_s(X)$ for the cipher value space induced by encoding the latent type X under secret s, that is, the image of $\mathrm{enc}(\cdot, \cdot)$ in $\{0,1\}^n$. When the secret is clear from context, we drop the subscript and write $\mathsf{C}(X)$. The same notation applies to latent functions: for $f : X \to Y$, the cipher map of f under secret s is written $\mathsf{C}_s(f) : \mathsf{C}_s(X) \to \mathsf{C}_s(Y)$. Under the random oracle model and the four properties developed in §4, $\mathsf{C}_s(\cdot)$ behaves as a functor from finite latent types to a category $\mathbf{Cipher}_s$ of cipher value spaces: it takes objects (types) to objects and morphisms (functions) to morphisms, and respects composition up to the error bound of §7 (exactly under the re-randomization condition, Definition 7.1). The categorical apparatus, including the rekeying functor $r_{s_1 \to s_2} : \mathbf{Cipher}_{s_1} \to \mathbf{Cipher}_{s_2}$ and its information-theoretic chain bound, is developed in companion work [30]; in this paper we use the notation as a clarifying shorthand without invoking the categorical machinery.

3.2 Definition

Definition 3.1 (Cipher map)**.** Let $f : X \to Y$ be a *latent function.* A *cipher map* for f is a tuple $(\hat{f}, \mathrm{enc}, \mathrm{dec}, s)$ where:

1. $\hat{f} : \{0,1\}^n \to \{0,1\}^n$ is a **total function** on n-bit strings;
2. $\mathrm{enc} : X \times \{0, \ldots, K(x)-1\} \to \{0,1\}^n$ is an encoding function, where $K(x) \geq 1$ is the number of encodings for element x;
3. $\mathrm{dec} : \{0,1\}^n \to Y \cup \{\perp\}$ is a decoding function;
4. s is a secret (the trapdoor) from which $\hat{f}$, enc, and dec are derived.

The untrusted machine holds $\hat{f}$. The trusted machine holds enc, dec, and s.

A cipher value $\mathrm{enc}(v, k)$ can be viewed as a cipher map for the constant function $f_v(x) = v$: under this view, cipher values and cipher maps are the same kind of object—a total function on bit strings—simplifying the untrusted machine's interface.

3.3 Two Construction Strategies

Cipher maps arise from two fundamentally different construction strategies:

Batch construction. The trusted machine invests computation upfront to find a seed s such that $\hat{f}$ satisfies correctness constraints for all $x \in X$ simultaneously. This gives tunable correctness parameter η and optimal space, but requires knowing X and f at construction time. The entropy cipher map (§6) is the general batch construction.

Online construction. The cipher map $\hat{f}$ is defined directly by the hash structure—no seed search is needed. The secret s is a hash key, not a searched seed. Operations are limited to those the hash structure supports algebraically (e.g., set union, intersection). Space and accuracy trade-offs are fixed by the hash width n. The trapdoor Boolean algebra (§9.3) is the online construction.

Both strategies produce objects satisfying Definition 3.1.

3.4 Construction Layers

The four properties (defined in §4) arise from three orthogonal type transformations applied to the latent function.

Layer 1: Undefined injection. Extend the domain X to $X \cup \{\perp_1, \ldots, \perp_N\}$ by adding N undefined elements. A function $f : X \to Y$ lifts to a partial function $\tilde{f}$ that is undefined on the $\perp_i$. Real elements become a fraction $|X|/(|X| + N)$ of the extended type. This controls the space parameter ε (the probability that random bits form a valid codeword).

Layer 2: Noise closure. Lift the partial function $\tilde{f}$ to a total function $\hat{f}$ by mapping undefined inputs to random outputs via cryptographic hash. The untrusted machine cannot distinguish real inputs from undefined inputs—both produce n-bit output. This yields **Property 1 (Totality)**.

Layer 3: Multiple representations. Replace each value x with $K(x) \geq 1$ distinct representations, keyed by the secret s. Representational equality implies value equality, but value equality does not imply representational equality. Assigning $K(x) \propto 1/D(x)$ equalizes frequencies. This yields **Property 2 (Representation Uniformity)**.

The combined type is cipher(noise(undef(X)))—an element with multiple representations, total evaluation, and diluted domain. Properties 3 (Correctness) and 4 (Composability) are emergent: correctness depends on the construction method; composability depends on the totality guarantee enabling chained evaluation.

These layers are conceptual scaffolding explaining *why* each property exists, not algebraic monads. Whether the layers satisfy formal monad laws in the approximate setting is an open question (see §9).

4 Four Properties

A cipher map $(\hat{f}, \text{enc}, \text{dec}, s)$ for $f : X \to Y$ is characterized by four properties parameterized by $(\eta, \varepsilon, \mu, \delta)$ (summarized in Table 1).

4.1 Property 1: Totality

Definition 4.1 (Totality)**.** The function $\hat{f} : \{0,1\}^n \to \{0,1\}^n$ is *total*: for every $c \in \{0,1\}^n$, $\hat{f}(c) \in \{0,1\}^n$. For c chosen uniformly at random from $\{0,1\}^n \setminus \text{Im}(\text{enc})$, the distribution of $\hat{f}(c)$ is uniform over $\{0,1\}^n$ under the random oracle model.

Totality ensures that the untrusted machine cannot distinguish a real query $\text{enc}(x, k)$ from a random filler string $c \leftarrow \{0,1\}^n$, because both produce n-bit outputs. If out-of-domain queries returned an error or $\perp$, the untrusted machine could identify real queries by observing which inputs produce valid output.

4.2 Property 2: Representation Uniformity (δ-bounded)

Definition 4.2 (Representation uniformity)**.** Let D be a distribution on X. Define the induced distribution on cipher values:

$$Q(c) = \sum_{x \in X} D(x) \cdot \frac{|\{k : \mathrm{enc}(x,k) = c\}|}{K(x)}. \tag{1}$$

The cipher map has δ-*representation uniformity* if

$$d_{\mathrm{TV}}(Q,\, \mathrm{Uniform}(\{0,1\}^n)) \le \delta, \tag{2}$$

where d_{TV} is total variation distance.

Total variation distance has the operational interpretation that no statistical test can distinguish Q from uniform with advantage exceeding δ. To achieve small δ, assign $K(x) \propto 1/D(x)$ encodings per value (homophonic substitution), so that frequent values get more representations.

Remark 4.1 (Marginal uniformity only). Representation uniformity bounds the *marginal* distribution of individual cipher values. It says nothing about *joint* distributions: an adversary observing sequences of cipher values can still detect correlations. See §8.1 for the encoding granularity trade-off.

4.3 Property 3: Correctness (η-bounded)

Definition 4.3 (Correctness)**.** For x chosen uniformly from X and k chosen uniformly from $\{0, \ldots, K(x) - 1\}$:

$$\Pr_{x,k}\big[\mathrm{dec}(\hat{f}(\mathrm{enc}(x,k))) \neq f(x)\big] \le \eta. \tag{3}$$

The parameter $\eta \in [0, 1)$ is a *construction* parameter: it reflects how many constraints the seed s must satisfy. Tolerating $\eta > 0$ makes the seed search faster and the structure smaller. Once built, the failing elements are deterministic for a given seed—the same elements always fail.

Remark 4.2 (Approximation as privacy). Nonzero η also provides privacy: a false positive creates plausible deniability, since the untrusted machine cannot distinguish "element is in the set" from "cipher map erred." With $\eta = 0$, the answer is deterministic; with $\eta > 0$, each answer carries ambiguity.

4.4 Property 4: Composability

Definition 4.4 (Composability)**.** The composition of two cipher maps is again a cipher map. Specifically, let $\mathsf{C}_s(f) : \mathsf{C}_s(X) \to \mathsf{C}_s(Y)$ be a cipher map for $f : X \to Y$ and $\mathsf{C}_s(g) : \mathsf{C}_s(Y) \to \mathsf{C}_s(Z)$ a cipher map for $g : Y \to Z$ (both under the same secret s, sharing encoding and decoding for type Y). Then

$$\mathsf{C}_s(g) \circ \mathsf{C}_s(f) \;:\; \mathsf{C}_s(X) \to \mathsf{C}_s(Z),$$

defined by $(\mathsf{C}_s(g) \circ \mathsf{C}_s(f))(c) = \mathsf{C}_s(g)(\mathsf{C}_s(f)(c))$, realizes a cipher map for $g \circ f : X \to Z$. In tuple form: if $(\hat{f}, \mathrm{enc}_f, \mathrm{dec}_f, s)$ and $(\hat{g}, \mathrm{enc}_g, \mathrm{dec}_g, s)$ are the underlying cipher maps for f and g, the composition has cipher function $\hat{g} \circ \hat{f}$, encoding enc_f, and decoding dec_g.

Remark 4.3 (Master secret vs. operational sub-derivations). Throughout this paper, "under secret s" refers to the master secret shared across the cipher map family. Each individual cipher map's hash randomization derives from s via a domain separator unique to the cipher map (e.g., the operational seed for $\mathsf{C}_s(f)$ is $op\_seed_f = \mathrm{HMAC}(s, \texttt{"f"})$). Under the random oracle model, distinct cipher maps under the same master secret have independent operational seeds; this independence is what makes the re-randomization condition (Definition 7.1) the typical case rather than an exception.

Composability is enabled by totality: since $\mathsf{C}_s(f)$ is total, its output is always a valid n-bit input to $\mathsf{C}_s(g)$. The untrusted machine can chain cipher maps without needing to decode intermediate results. In functorial language, $\mathsf{C}_s(\cdot)$ takes the latent composition $g \circ f$ to the cipher composition $\mathsf{C}_s(g) \circ \mathsf{C}_s(f)$, agreeing exactly under the re-randomization condition (Definition 7.1) and up to the bound of Theorem 7.2 otherwise. Composing across different secrets requires the rekeying functor of [30]; we restrict attention to single-secret composition in this paper.

Theorem 4.1 (Composition correctness)**.** *Under the hypotheses of Definition 4.4, if $\hat{f}$ has correctness η_f and $\hat{g}$ has correctness η_g, then $\hat{g} \circ \hat{f}$ has correctness*

$$\eta_{g \circ f} \leq \eta_f + \eta_g - \eta_f \eta_g = 1 - (1 - \eta_f)(1 - \eta_g),$$

with equality under the re-randomization condition (see §7, Theorem 7.2 and Definition 7.1).

Remark 4.4 (Noise under composition). Noise input to $\hat{f}$ produces random output (by totality). With probability ε, that random output happens to be a valid codeword for $\hat{g}$. We do not try to control noise output—the cipher map is just a hash. This is a feature: valid-looking outputs do not prove valid inputs.

4.5 Parameter Decomposition

Table 1: Cipher map parameters.

Parameter	Range	Controls	Determined by
η (correctness)	$[0, 1)$	Fraction of wrong answers	Construction (seed search)
ε (noise decode)	$(0, 1)$	Pr[random bits valid]	Encoding scheme
μ (value cost)	$(0, \infty)$	Bits per element for values	$\mu = H(Y)$
δ (uniformity)	$[0, 1]$	TV distance from uniform	Multiplicity $K(x)$
$K(x)$ (multiplicity)	$\{1, 2, \ldots\}$	Encodings per element	Set to $\propto 1/D(x)$
p (entanglement)	$\{1, \ldots, k\}$	Encoding granularity	Correlation structure (§8.1)

The space per element decomposes as:

$$\text{bits/element} = -\log_2 \varepsilon + \mu = -\log_2 \varepsilon + H(Y),$$

where $-\log_2 \varepsilon$ bits distinguish signal from noise and $H(Y)$ bits encode the function value. This decomposition is information-theoretic.

5 The Trusted/Untrusted Machine Model

The four properties take operational meaning through a two-party model. Throughout, we adopt the standard adversary convention from quantitative information flow [25]: the untrusted machine U

is *honest-but-curious*, faithfully evaluating $\hat{f}$ on every input it receives but attempting to learn information about the latent function f, the domain X, the codomain Y, or specific values from the observed input/output trace. Active deviations (returning incorrect results, refusing queries, inserting its own queries) are out of scope for this paper; they are addressable by orthogonal verification or auditing layers, but the parameterized leakage we analyze is the same.

Definition 5.1 (Trusted machine T)**.** The trusted machine T holds: the seed s (trapdoor), the encoding function enc, the decoding function dec, and knowledge of which queries are real vs. filler.

T can:

1. Encode plaintext values: $x \mapsto \mathrm{enc}(x, k)$ for chosen k.
2. Decode cipher results: $r \mapsto \mathrm{dec}(r)$.
3. Inject noise: generate random $c \leftarrow \{0,1\}^n$ as filler queries.
4. Verify results: check $\mathrm{dec}(\hat{f}(\mathrm{enc}(x, k))) = f(x)$.
5. Reseed: construct a new cipher map with fresh s' when leakage accumulates.

Definition 5.2 (Untrusted machine U)**.** The untrusted machine U holds: the cipher map $\hat{f}$ (a total function on bit strings) and a collection of cipher values (encodings plus filler, indistinguishable to U).

U can:

1. Evaluate cipher maps: $c \mapsto \hat{f}(c)$ for any $c \in \{0,1\}^n$.
2. Return results to T.
3. Compute arbitrary functions of its observed trace $\tau = ((c_i, \hat{f}(c_i)))_{i \in [N]}$.

U's view of the latent function f is the trace τ together with the cipher map $\hat{f}$. We analyze what U can learn from this view as a quantitative leakage measure rather than as a binary security event.

5.1 The Confidentiality Measure

We adopt the *entropy ratio* of the companion entropy ratio paper [29, §4].

Definition 5.3 (Entropy ratio)**.** Let Q denote the distribution of cipher values that U observes (the marginal of the trace's input column). The *entropy ratio* of U's view is

$$e(\hat{f}, \tau) \;=\; H(Q) \,/\, n,$$

where $H(Q)$ is the Shannon entropy of Q and n is the cipher value space dimension (so the maximum-entropy reference is the uniform distribution on $\{0,1\}^n$, with entropy n bits). The entropy ratio satisfies $0 \le e \le 1$: $e = 1$ corresponds to a uniformly distributed cipher value stream (no marginal leakage from the cipher value distribution alone), and $e \to 0$ corresponds to a degenerate stream concentrated on few values.

This normalized form of Shannon leakage is standard in the quantitative information flow tradition [25, 2] and the entropic security literature [12]. We use Shannon entropy rather than min-entropy because the relevant adversary goal in this paper is *average-case* information recovery from many cipher value observations (where Shannon entropy gives the information-theoretic ceiling on the adversary's expected gain), not single-try guessing (where min-entropy is more natural). Settings where min-entropy is preferred (one-shot key recovery, cryptographic guessing games) are treated in the companion work [29, §4].

We record only the bridge to the cipher map parameters here; formal definitions, decomposition into multiple components, and discussion of when this measure suffices versus when other QIF measures (min-entropy leakage, g-leakage) apply, all appear in [29, §4].

Proposition 5.1 (Confidentiality bound for cipher maps)**.** *Let $(\hat{f}, \mathrm{enc}, \mathrm{dec}, s)$ be a cipher map with representation-uniformity parameter $\delta \leq 1/2$ over an n-bit cipher value space (Property 2). Then the entropy ratio of U's view satisfies*

$$e(\hat{f}, \tau) \;\geq\; 1 - \delta - h_2(\delta)/n, \tag{4}$$

where $h_2(\delta) = -\delta \log_2 \delta - (1-\delta)\log_2(1-\delta)$ is the binary entropy. The condition $\delta \leq 1/2$ is the standard applicability range of the Fannes–Audenaert continuity inequality; for $\delta > 1/2$ the bound is trivial ($e \geq 0$).

Proof sketch. Property 2 gives $d_{\mathrm{TV}}(Q, \mathrm{Uniform}) \leq \delta$ where Q is the cipher value distribution induced by enc on the $\{0,1\}^n$ ambient space. Applying the Fannes–Audenaert continuity inequality with reference distribution $\mathrm{Uniform}(\{0,1\}^n)$ (entropy n) gives $H(Q) \geq n(1-\delta) - h_2(\delta)$, and dividing by n yields (4). Full proof and discussion of tightness in [29, Theorem 4.1, part 3]. The bound applies specifically to the cipher value marginal; downstream bounds on what U can learn about the latent function f or the latent input X follow by data-processing arguments (§5.2). □

Remark 5.1 (The role of δ). Proposition 5.1 reduces confidentiality engineering for cipher maps to the concrete problem of minimizing δ. Three mechanisms reduce δ, each with explicit costs analyzed in the companion work: noise injection (bandwidth cost), multiple representations $K(x) > 1$ (storage cost; Simmons homophonic prescription), and joint encoding granularity (storage cost growing as $|Y|^k$). We discuss the multiplicity mechanism in §8.1; the others are out of scope for this paper.

5.2 Operational Consequences

From the bound (4) together with the four properties, several operational consequences follow that may be more familiar to readers expecting binary security claims. These are not separate security guarantees; they are concrete restatements of the entropy ratio bound under specific adversary goals.

1. *Decoding requires the trapdoor.* Recovering dec from $\hat{f}$ requires inverting the cryptographic hash h used in the construction (§6). Under the random oracle model (Preliminaries, §3), this is computationally infeasible.

2. *Real and filler queries are δ-equivocal in distribution (Properties 1 and 2 together).* Both are observed as n-bit cipher values, and the cipher value distribution is δ-close to uniform by Property 2. Consequently, no statistical test distinguishes real from filler with advantage exceeding $\delta/2$ (Le Cam's two-point lemma applied to the cipher value marginals; we report δ as a conservative upper bound throughout to match the total-variation normalization).

3. *Domain identification is bounded by δ (Property 2).* For any binary discrimination question U poses (e.g., "is this cipher value a real domain element or noise?"), U's advantage over random guessing is at most δ, by the data-processing inequality applied to Property 2's TV bound. Aggregated over N independent observations, U's advantage on a recovery game grows no faster than the entropy ratio bound of Proposition 5.1 permits.

4. *Function-value leakage is bounded by the entropy ratio.* For any function $g(f)$ that U wishes to extract, U's information gain (mutual information between the trace and $g(f)$) is bounded above by $n(1 - e(\hat{f}, \tau))$ bits, by the data-processing inequality applied to Proposition 5.1. Translating this bit bound into a guessing-advantage bound depends on the prior over $g(f)$ and is treated for specific adversary games in [29, §5–6].

Protocol. The information flow is:

1. T encodes inputs: $\text{enc}(x_1, k_1), \ldots, \text{enc}(x_m, k_m)$.
2. T generates filler: $c_1, \ldots, c_r \leftarrow \{0,1\}^n$.
3. T sends all cipher values (real plus filler, shuffled) to U.
4. U evaluates $\hat{f}$ on each cipher value and returns results.
5. T decodes the results it cares about and discards filler results.

Table 2: Quantitative confidentiality bounds against U (honest-but-curious). The entropy ratio e is the unifying measure; Proposition 5.1 ties it to δ via the Fannes–Audenaert continuity inequality.

Adversary goal	Bound	Mechanism
Distinguish real from filler	$d_{\text{TV}} \leq \delta$	Totality + Property 2
Frequency-analyze cipher values	$d_{\text{TV}} \leq \delta$	Property 2 (uniformity)
Recover plaintext value $f(x)$	$1 - e$ advantage	Prop. 5.1 via δ
Test membership $x \in X$	δ-bounded	Totality + Property 2
Detect cross-query correlations	**Not bounded**	Marginal uniformity only (§8.2)

The last row is the honest limitation: marginal δ-uniformity per cipher map does not bound joint correlations across multiple cipher maps sharing inputs. The compositional leakage analysis in §8.2 treats this case; the encoding granularity parameter p controls the trade-off between space and joint correlation hiding.

6 The Batch Construction

The batch construction requires knowing X and f at construction time and invests computation in a seed search to find a cipher map satisfying correctness constraints for all stored elements. This section develops the batch construction from the information-theoretic lower bound through the acceptance predicate framework, culminating in the entropy cipher map—the general batch cipher map for arbitrary $f : X \to Y$.

6.1 Information-Theoretic Lower Bound

Theorem 6.1 (Lower bound)**.** *Any data structure representing an approximate map $\hat{f}$ for $f : X \to Y$ with noise decode probability ε and correctness $\eta = 0$ over n stored elements requires at least*

$$n\big(-\log_2 \varepsilon + H(Y)\big) \text{ bits}, \tag{5}$$

or equivalently $-\log_2 \varepsilon + H(Y)$ bits per element.

Proof. The bound decomposes into two independent information requirements.

Membership component. Consider the membership predicate $\mathrm{dec}(\hat{f}(c)) \neq \perp$. For a universe U with $|U| \gg n$, the structure must distinguish n stored elements from $|U| - n$ non-elements, with at most an ε fraction of non-elements decoding to valid output. There are $\binom{|U|}{n}$ possible sets of size n from a universe U. Any representation must distinguish all such sets, requiring at least $\log_2 \binom{|U|}{n}$ bits. For $|U| \gg n$, Stirling's approximation $\log_2 \binom{|U|}{n} = n \log_2(|U|/n) + n \log_2 e + O(\log n)$ gives $\log_2 \binom{|U|}{n} \approx n \log_2(|U|/n)$ bits to leading order in $|U|/n$, or $\log_2(|U|/n)$ bits per element. We drop the $n \log_2 e \approx 1.44n$ subleading term: it is independent of $|U|$ and adds at most a constant overhead per element, which the matching achievability construction (Theorem 6.2) matches asymptotically. The noise-decode probability ε is the fraction of bit strings in $\{0,1\}^n$ that decode to some valid output value: under any membership-faithful encoding, exactly the n stored elements decode validly out of the $|U|$ universe inputs, so $\varepsilon = n/|U|$ when the universe is identified with the n-bit cipher value space. Substituting yields $n \log_2(1/\varepsilon)$ bits.

Value component. Independently of membership, the structure must encode the function values $(f(x_1), \ldots, f(x_n))$. By Shannon's source coding theorem [23], encoding n independent draws from the distribution (p_y) requires at least $nH(Y)$ bits.

Since membership and value encoding are jointly necessary, the total requirement is $n(-\log_2 \varepsilon + H(Y))$ bits. The additivity holds because the membership bits and value-encoding bits convey independent information: by the entropy chain rule, $H(\text{membership}, \text{value}) = H(\text{membership}) + H(\text{value} \mid \text{membership})$, and conditioning on membership (i.e., knowing which elements are stored) does not reduce the entropy of the value assignment, since the values $(f(x_1), \ldots, f(x_n))$ are an independent degree of freedom. □

6.2 Acceptance Predicates

The construction depends on a rule for deciding whether a hash value encodes a particular output. We abstract this as an *acceptance predicate*.

Definition 6.1 (Acceptance predicate)**.** An acceptance predicate for codomain Y is a family of disjoint sets $\{A(y) \subseteq \{0,1\}^n : y \in Y\}$. The decoder is $\mathrm{dec}(\text{hash}) = y$ if hash $\in A(y)$, and $\perp$ otherwise. The acceptance probability for value y is $\alpha(y) = |A(y)|/2^n$, and the noise-decode probability is $\varepsilon = \sum_y \alpha(y) = |\bigcup_y A(y)|/2^n$.

Remark 6.1 (XOR-scrambled decode). The decode function applies a secret-derived XOR mask before checking acceptance boundaries: $\mathrm{dec}(r)$ checks whether $r \oplus s_{\text{key}}$ falls in the acceptance region for some y. Without the scramble key, the mapping from hash values to output values is pseudorandom; contiguous acceptance regions in the scrambled space appear scattered in the raw hash space. This prevents an adversary observing hash outputs from inferring which outputs correspond to which values, even if the acceptance regions are contiguous.

Different predicates give different space and construction-time trade-offs:

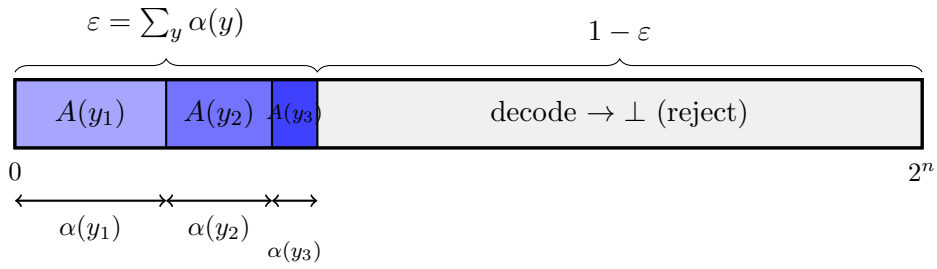

Figure 1: Acceptance predicate partition of the n-bit hash space. Each value $y \in Y$ is assigned an acceptance set $A(y)$ with probability $\alpha(y) = |A(y)|/2^n$. Under Shannon-optimal allocation $\alpha(y) = \varepsilon \cdot p_y$ (where $p_y = \Pr[f(X) = y]$), region widths are proportional to value frequencies: the dominant value y_1 occupies the largest region. A hash falling in $A(y)$ decodes to y; a hash outside the accepted region (total measure $1-\varepsilon$) decodes to $\perp$. The Shannon-frequency duality (§6.2) holds because the same allocation that minimizes the per-element bit cost ($-\log_2 \varepsilon + H(Y)$) also makes the cipher value distribution match the latent value distribution, defeating frequency analysis.

- **Prefix-free code.** $A(y) = \{c \in \{0,1\}^n : c \text{ starts with codeword}(y)\}$. Acceptance probability $\alpha(y) = 2^{-|\text{code}(y)|}$. Shannon-optimal when $|\text{code}(y)| \approx -\log_2 p_y$. This is the entropy cipher map construction.
- **Threshold.** $A(y) = \{c : c < t(y)\}$ for thresholds $0 = t_0 < t_1 < \cdots$. Acceptance probability $\alpha(y) = (t_{y+1} - t_y)/2^n$. Allows fine-grained control of per-value acceptance rates.
- **Range partition.** Partition $[0, 2^n)$ into contiguous ranges $[l_y, l_y + w_y)$. Setting $w_y \propto p_y$ achieves Shannon-optimal allocation with uniform acceptance probability.

All three achieve the same space bound $-\log_2 \varepsilon + H(Y)$ bits per element when acceptance probabilities are Shannon-optimal ($\alpha(y) \propto p_y$). They differ in construction time and implementation simplicity.

Shannon-optimal allocation and frequency hiding. Setting $\alpha(y) \propto \Pr[f(X) = y]$ simultaneously achieves two goals (Figure 1). First, it minimizes space: the per-element encoding cost $-\log_2 \alpha(y) \approx -\log_2 p_y$ is the Shannon information content of value y, and the average cost $\sum_y p_y(-\log_2 p_y) = H(Y)$ is the entropy of the output distribution. Second, it hides value frequencies: noise queries (random hash values) hit acceptance set $A(y)$ with probability $\alpha(y) \propto p_y$—the same distribution as real queries. An adversary observing output values cannot distinguish whether a result came from a real query or noise, because the output frequency profile is identical in both cases.

These are not two independent optimizations that happen to coincide. Shannon-optimal coding IS frequency hiding: matching the acceptance distribution to the output distribution simultaneously minimizes space and maximizes indistinguishability. Any deviation from $\alpha(y) \propto p_y$ both wastes space (encoding rare values with fewer bits than their information content requires) and leaks information (the frequency profile of outputs differs between real and noise queries).

6.3 Construction Algorithm

Algorithm 1 searches for a seed ℓ such that at most $\lfloor \eta \cdot m \rfloor$ elements of M fail the acceptance predicate. When $\eta = 0$, the first failure triggers rejection (the exact case). Query evaluation computes $\hat{f}(c) = h(\ell) \oplus h(c)$ and decodes via A.

Algorithm 1 Batch Cipher Map Construction

```
1:  function BuildCipherMap(M, A, h, η)        ▷ A: acceptance predicate, η: target error rate
2:      m ← |M|
3:      ℓ ← 0
4:      while true do
5:          Shuffle(M)                          ▷ random order for unbiased early stopping
6:          fail ← 0;  pass ← 0
7:          for (x, y) ∈ M do
8:              hash ← h(ℓ) ⊕ h(x)
9:              if hash ∈ A(y) then
10:                 pass ← pass + 1
11:                 if pass ≥ ⌈(1 − η) m⌉ then
12:                     return CipherMap(h, A, ℓ)                    ▷ early accept
13:                 end if
14:             else
15:                 fail ← fail + 1
16:                 if fail > ⌊η m⌋ then
17:                     break                                        ▷ early reject
18:                 end if
19:             end if
20:         end for
21:         ℓ ← ℓ + 1
22:     end while
23: end function
```

Two early-stopping rules avoid checking all m elements:

- **Early reject:** if failures exceed $\lfloor \eta\, m \rfloor$, no remaining successes can help.
- **Early accept:** if successes reach $\lceil (1-\eta)\, m \rceil$, the error budget is satisfied regardless of remaining elements.

Shuffling M each iteration ensures both rules trigger as early as possible: failures are not clustered at the end by a fixed ordering.

6.4 Space Optimality

Theorem 6.2 (Information-theoretic space complexity)**.** *The batch construction has information-theoretic space complexity*

$$(1-\eta)\big(-\log_2 \varepsilon + \mu\big) \text{ bits per element}, \tag{6}$$

where $\mu = H(Y)$ is the mean encoding length. This is the information content of the $(1-\eta)n$ correctly encoded elements. When $\eta = 0$, the bound matches the lower bound of Theorem 6.1.

Proof. Step 1: False negative reduction. When false negatives are permitted at rate η, only $(1-\eta)n$ of the n elements must decode correctly; the failing elements carry no information about the latent function and contribute zero to the information content of the construction.

Step 2: Per-element seed-table storage cost. For each correctly stored element x_i with target value y_i, the construction must record which of the $1/\alpha(y_i)$ candidate seeds satisfies the acceptance constraint $h(\ell) \oplus h(x_i) \in A(y_i)$ (Definition 6.1). Under the random oracle model with Shannon-optimal allocation $\alpha(y) = \varepsilon \cdot p_y$ (proportional to value frequency, normalized so $\sum_y \alpha(y) = \varepsilon$), the seed-table storage cost per element is $-\log_2 \alpha(y_i) = -\log_2 \varepsilon - \log_2 p_y$ bits. Per-element storage costs sum across the $(1-\eta)n$ correctly encoded elements (by independence of the per-element acceptance events under the random oracle model). We emphasise that this is the storage cost (bits to record the chosen seed), separate from the construction-time cost (number of seed trials, which scales as $1/\alpha(y_i)$ per element and is analysed in §6.5). Averaging the storage cost over the value distribution gives

$$\mathbb{E}_{y\sim p}\big[-\log_2 \alpha(y)\big] \;=\; \mathbb{E}_{y\sim p}\big[-\log_2 \varepsilon - \log_2 p_y\big] \;=\; -\log_2 \varepsilon \;+\; H(Y).$$

The two terms have distinct interpretations: $-\log_2 \varepsilon$ is the constant cost of distinguishing a valid encoding from one of the $2^n(1-\varepsilon)$ noise-decoding bit strings, and $H(Y) = \mu$ is the amortized cost of identifying *which* value an encoding carries. Choosing $\alpha(y) \propto p_y$ achieves the entropy lower bound on the second term while pinning the first.

Step 3: Combining. Each correctly encoded element carries $-\log_2 \varepsilon + \mu$ bits of information, and $(1-\eta)n$ such elements are encoded, giving total information content $(1-\eta)n(-\log_2 \varepsilon + \mu)$ bits. □

Remark 6.2 (Information-theoretic vs. physical storage). The $(1-\eta)$ factor is an information-theoretic statement about useful content, not an automatic reduction in physical storage. In the two-level hash construction (Algorithm 1), the seed table contains the same number of entries regardless of η: the ηn failing elements still occupy slots, but their seeds are "don't cares." Physical storage matches the bound when the seed table is entropy-coded; uncompressed storage is $-\log_2 \varepsilon + \mu$ bits per element regardless of η. The η parameter primarily reduces construction difficulty (fewer constraints to satisfy), not raw storage.

Corollary 6.3 (Asymptotic optimality)**.** *When $\eta = 0$, the batch construction achieves $-\log_2 \varepsilon + H(Y)$ bits per element, matching the information-theoretic lower bound.*

6.5 Construction Time and Bucketing

Proposition 6.4 (Per-seed success probability)**.** *For m stored elements with acceptance predicate A and target error rate η, each element independently passes with probability $\alpha(y_i)$ under the random oracle model. The number of failures F follows a Poisson binomial distribution, and the seed is accepted when $F \leq \lfloor \eta m \rfloor$.*

Special cases:

- $\eta = 0$*:* $p = \prod_{i=1}^{m} \alpha(y_i)$ *(all must pass).*
- *Uniform* α*:* $p = \Pr[\mathrm{Bin}(m, 1-\alpha) \leq \lfloor \eta m \rfloor]$.

The expected number of seeds tried is $1/p$.

Proof. Under the random oracle model, the hash values $h(\ell) \oplus h(x_i)$ for distinct x_i are independent. Element x_i fails with probability $1 - \alpha(y_i)$. The failure count $F = \sum_i \mathbf{1}[\mathrm{hash}_i \notin A(y_i)]$ is a sum of independent Bernoulli variables. The seed is accepted when $F \leq \lfloor \eta m \rfloor$, giving the Poisson binomial CDF. For uniform α, this reduces to the binomial CDF. □

The effect of $\eta > 0$ on construction time is dramatic. For uniform α and $m = 100$:

α	$\eta = 0$	$\eta = 0.01$	$\eta = 0.05$
0.5	2^{100}	2^{93}	2^{74}
0.9	3.8×10^4	3.1×10^3	17
0.99	2.7	1.5	1.0

When η exceeds the expected failure rate $1 - \alpha$, the binomial tail concentrates and construction becomes near-certain on the first seed. The transition is sharp: a few percent of tolerable error can reduce construction time by tens of orders of magnitude.

Bucketed construction. The intractability of the global seed search is not inherent. Partition the m elements into k buckets (e.g., by $h(x) \bmod k$) and find a separate seed ℓ_j for each bucket.

Proposition 6.5 (Bucketed construction time ($\eta = 0$))**.** *With k buckets of expected size m/k and $\eta = 0$, the expected total construction time is*

$$T(k) = k \cdot \left(\frac{1}{\bar{\alpha}}\right)^{m/k}, \tag{7}$$

where $\bar{\alpha} = \left(\prod_i \alpha(y_i)\right)^{1/m}$ is the geometric mean acceptance probability.

Proof. Each bucket has m/k elements (in expectation). The per-bucket success probability is $\bar{\alpha}^{m/k}$ by Proposition 6.4. Expected trials per bucket: $(1/\bar{\alpha})^{m/k}$. Summing over k independent buckets gives the result. □

The trade-off between construction time and space is controlled by k:

Buckets k	Seeds stored	Construction time	Space overhead
1 (global)	1	$(1/\bar{\alpha})^m$	$O(\log m)/m$ bits/elem
$\sqrt{m}$	$\sqrt{m}$	$\sqrt{m} \cdot (1/\bar{\alpha})^{\sqrt{m}}$	$O(\log m/\sqrt{m})$ bits/elem
m (per-element)	m	$m/\bar{\alpha}$ (linear)	$O(\log(1/\bar{\alpha}))$ bits/elem

At $k = m$, each bucket has one element, construction is linear in m, and the overhead is $\log_2(1/\bar{\alpha})$ bits per element for the per-element seed—the same order as the hash width. This is the regime of practical perfect hash functions [3].

An alternative construction uses perfect hash functions (PHFs) to assign elements to slots in $O(m)$ time, avoiding the seed search entirely. A PHF maps the m cipher keys to m distinct slots; each slot stores the XOR-masked value encoding. Construction time drops from exponential in bucket size to linear in m. A reference implementation using a RecSplit-family PHF [13] achieves ~ 713 documents per second on the full 20 Newsgroups corpus (18,266 documents, 58,903 words; 25.6 s wall-clock), compared to approximately 10 documents per second with the seed search construction. Detailed construction-time scaling appears in §10.3.

6.6 Instantiations

6.6.1 Entropy Cipher Map

The *entropy cipher map* is the batch construction (Algorithm 1) instantiated with a prefix-free acceptance predicate. It is the general batch cipher map for arbitrary $f : X \to Y$, achieving Shannon-entropy-optimal value encoding.

Latent function. Arbitrary $f : X \to Y$ where Y is finite.

Construction. Assign a prefix-free code $C_y \subseteq \{0,1\}^*$ for each $y \in Y$, where the fraction of hash space covered by C_y is proportional to $\Pr_D[f(X) = y]$. Find a seed s (via two-level hashing, §6.5) such that

$$h(x\|s) \in C_{f(x)} \quad \text{for all } x \in X.$$

Evaluation: compute $h(c\|s)$ and decode via the prefix-free code.

Two-level hash (practical algorithm).

1. A primary hash maps x to a bucket index $j \in \{0, \ldots, b-1\}$.
2. For each bucket, find a local seed s_j such that $h(x\|s_j)$ decodes to $f(x)$ for all x in bucket j.
3. Store s_j in a seed table $H[j]$.
4. Query: compute bucket j (1 hash), evaluate $h(x\|H[j])$ and decode (1 hash). Total: $O(1)$ hash operations.

Mapping to cipher map abstraction.

- $\hat{f}(c) = h(c\|H[\text{bucket}(c)])$, total function.
- $\text{enc}(x, 0) = x$ (simplest version; $K(x) = 1$).
- $\text{dec}(r) = y$ if $r \in C_y$ for some y, else $\perp$.

Parameter instantiation.

Parameter	Value	Justification
η	Tunable	Fraction of elements failing acceptance predicate
ε	Code-dependent	$\Pr[\text{random bits} \in \bigcup_y C_y]$
μ	$H(Y)$	Shannon entropy of output distribution
δ	From code assignment	If $\lvert C_y\rvert \propto \Pr[f(X) = y]$, approaches uniform
Space	$-\log_2 \varepsilon + H(Y)$ bits/element	

Property status. Totality: **yes**. Representation uniformity: **achievable** (by assigning multiple codes proportional to output frequency). Correctness: **yes** ($\eta \approx 0$, tunable via Algorithm 1). Composability: **yes** (output is an n-bit string; serves as input to another cipher map).

Proposition 6.6 (Entropy cipher map space)**.** *The space per element of an entropy cipher map for $f : X \to Y$ with output distribution $(p_y)_{y \in Y}$ equals $-\log_2 \varepsilon + H(Y)$ bits, where $H(Y) = -\sum_y p_y \log_2 p_y$.*

Proof. The term $-\log_2 \varepsilon$ is the per-element cost of distinguishing signal from noise: each stored element requires that its hash prefix match a valid codeword, which occurs with probability ε for random inputs. The term $H(Y)$ is the expected code length under Shannon-optimal prefix-free coding, which equals the entropy of the output distribution by Shannon's source coding theorem [23]. These two components are additive because the noise-rejection bits and the value-encoding bits occupy disjoint portions of the hash output. □

Remark 6.3 (Set membership as a cipher map). Set membership $\mathbf{1}_A : X \to \text{Bool}$ is a cipher map with $Y = \{\mathit{true}, \mathit{false}\}$. The acceptance predicate partitions hash space into $A(\mathit{true})$ and $A(\mathit{false})$, with both outputs appearing as opaque bit strings. Shannon-optimal allocation sets $\lvert A(\mathit{true})\rvert \propto \Pr[x \in A]$ and $\lvert A(\mathit{false})\rvert \propto \Pr[x \notin A]$. For members, the seed search ensures $h(\ell) \oplus h(x) \in A(\mathit{true})$. For non-members, the hash lands in $A(\mathit{false})$ with probability $1 - \varepsilon$, providing both correct classification and output-frequency hiding. Space per element: $-\log_2 \varepsilon + H(\text{Bool})$ bits, where $H(\text{Bool}) = -p \log_2 p - (1-p) \log_2 (1-p)$ for $p = \Pr[x \in A]$.

The traditional Bloom filter [7] tests membership but reveals the raw Boolean result. The cipher map version hides both the input (via enc) and the output (via the acceptance predicate), at the cost of storing per-element seeds and accepting a noise-decode probability ε.

7 Composition

7.1 Warm-Up: The AND Gate

Before proving the general composition theorem, we analyze a concrete case: the AND gate operating on two independent Bernoulli Boolean values B_1, B_2 with correctness probabilities $p_1 = 1 - \eta_1$ and $p_2 = 1 - \eta_2$.

Proposition 7.1 (AND gate correctness)**.** *The correctness probability of* $\text{AND}(B_1, B_2)$ *as an approximation of* $\text{AND}(x_1, x_2)$ *depends on the latent inputs:*

x_1	x_2	AND(x_1, x_2)	Pr[*correct*]
1	1	1	$p_1 p_2$
1	0	0	$1 - p_1(1 - p_2)$
0	1	0	$1 - p_2(1 - p_1)$
0	0	0	$p_1 + p_2 - p_1 p_2$

Proof. We verify the $(1, 0)$ case; the others are analogous. When $x_1 = 1$, $x_2 = 0$, the correct output is 0. The output is wrong ($= 1$) only when $B_1 = 1$ (correct, probability p_1) *and* $B_2 = 1$ (incorrect, probability $1 - p_2$). So $\Pr[\text{wrong}] = p_1(1 - p_2)$ and $\Pr[\text{correct}] = 1 - p_1(1 - p_2) = 1 - p_1 + p_1 p_2$. □

The output has a *four-case correctness profile*: four distinct correctness probabilities, one per input pair $(a, b) \in \{0, 1\}^2$, despite the Boolean output taking only two values. Rather than tracking all four cases separately, interval arithmetic propagates a bound $[\min_{\text{cases}} p_{\text{correct}},\ \max_{\text{cases}} p_{\text{correct}}]$. For AND with $p_1, p_2 \in (0.5, 1)$: minimum $p_1 p_2$ (case $(1, 1)$); maximum $p_1 + p_2 - p_1 p_2$ (case $(0, 0)$).

7.2 General Composition Theorem

Definition 7.1 (Re-randomization condition)**.** A composition $\hat{g} \circ \hat{f}$ satisfies the *re-randomization condition* when (i) $\hat{f}$ and $\hat{g}$ are constructed with independent operational sub-derivations of the master secret (automatic under the random oracle model with distinct domain separators per cipher map; see Remark 4.3), and (ii) conditional on $\hat{f}$ producing a particular output, $\hat{g}$'s correctness on that output is independent of whether $\hat{f}$ was correct. Operationally, this requires that the encoded output of $\hat{f}$ is statistically indistinguishable from a freshly drawn cipher value at $\hat{g}$'s input, so that $\hat{g}$'s acceptance behavior cannot be correlated with $\hat{f}$'s error events. Domain-separated sub-keys suffice for (i); fresh per-call multiplicities $K(x) > 1$ or noise injection between maps are mechanisms for (ii).

Theorem 7.2 (Composition correctness)**.** *Let $\hat{f}$ be a cipher map for $f : X \to Y$ with correctness η_f, and $\hat{g}$ a cipher map for $g : Y \to Z$ with correctness η_g. Then $\hat{g} \circ \hat{f}$ is a cipher map for $g \circ f$ with correctness*

$$\eta_{g \circ f} \leq \eta_f + \eta_g - \eta_f \eta_g = 1 - (1 - \eta_f)(1 - \eta_g), \tag{8}$$

with equality under the re-randomization condition (Definition 7.1).

Proof. The composition is incorrect when at least one map errs. Let $A = \{\hat{f} \text{ errs}\}$ and $B = \{\hat{g} \text{ errs}\}$. By inclusion-exclusion, $\Pr[A \cup B] = \Pr[A] + \Pr[B] - \Pr[A \cap B]$. Note that the bound $\Pr[A \cup B]$ is pessimistic: it counts every case in which at least one map errs as a composition error, but occasional error cancellation occurs when $\hat{f}$ produces a wrong intermediate value $y' \neq f(x)$ on which $\hat{g}$ nonetheless evaluates to $g(f(x))$. Such cancellations are rare in well-chosen codomains and we do not attempt to credit them in the bound below.

Loose bound. Dropping $\Pr[A \cap B] \geq 0$ gives the union bound $\eta_{g \circ f} \leq \eta_f + \eta_g$.

Under re-randomization. If the errors are independent ($\Pr[A \cap B] = \eta_f \eta_g$), inclusion-exclusion gives $\eta_{g \circ f} = \eta_f + \eta_g - \eta_f \eta_g = 1 - (1 - \eta_f)(1 - \eta_g)$.

In practice, $\hat{f}$ produces a specific encoding of $f(x)$, not a uniformly random one. If $\hat{g}$'s correctness depends on which encoding it receives, the errors may be positively correlated ($\Pr[A \cap B] \leq \eta_f \eta_g$), making the bound $\eta_f + \eta_g - \eta_f \eta_g$ conservative. For specific circuits, the case-by-case interval arithmetic from §7.1 gives tighter bounds. □

Remark 7.1 (Garbage propagation)*.* When $\hat{f}$ produces an incorrect output on some input, the value fed to $\hat{g}$ is effectively random. Since $\hat{g}$ is a total function, it produces *some* output on this random

input, which is correct with probability at most ε_g (the noise-decode probability of $\hat{g}$). Thus the bound $\eta_{\text{total}} \leq \eta_f + \eta_g - \eta_f\eta_g$ is an upper bound; the actual error rate may be marginally lower due to accidental correctness on garbage inputs. For small η_f, η_g, this distinction is negligible.

7.3 Composition Chains

Corollary 7.3 (Chain composition)**.** *For a chain of m cipher maps $\hat{f}_1, \ldots, \hat{f}_m$:*

$$\eta_{\text{total}} \leq 1 - \prod_{i=1}^{m}(1 - \eta_i), \tag{9}$$

with equality under the re-randomization condition (Definition 7.1) applied pairwise along the chain. If all $\eta_i = \eta$:

$$\eta_{\text{total}} \leq 1 - (1 - \eta)^m \approx m\eta \quad \textit{for small } \eta. \tag{10}$$

Proof. By induction on Theorem 7.2. The base case $m = 2$ is the theorem. For the inductive step, the composition of $m - 1$ maps has correctness bounded by $1 - \prod_{i=1}^{m-1}(1 - \eta_i)$; composing with $\hat{f}_m$ and applying Theorem 7.2 again gives $\eta_{\text{total}} \leq 1 - \prod_{i=1}^{m-1}(1 - \eta_i) \cdot (1 - \eta_m) = 1 - \prod_{i=1}^{m}(1 - \eta_i)$. The approximation follows from $\ln(1 - \eta) \approx -\eta$ for small η, giving $(1 - \eta)^m \approx e^{-m\eta} \approx 1 - m\eta$. □

Example 7.1. A circuit of 100 cipher maps each with $\eta = 10^{-6}$: $\eta_{\text{total}} \leq 1 - (1 - 10^{-6})^{100} \approx 10^{-4}$ (equality under re-randomization).

7.4 Error Accumulation by Gate Type

For specific circuits, the AND gate analysis (Proposition 7.1) can be extended to OR, NOT, and arbitrary Boolean gates. Each gate type induces a different case table. Rather than tracking exact correctness per case, interval arithmetic propagates bounds: at each gate, compute $[\eta_{\min}, \eta_{\max}]$ from the input intervals. This gives input-dependent bounds tighter than the worst-case Theorem 7.2.

8 Representation Uniformity and Encoding Granularity

We measure representation uniformity using total variation distance d_{TV}, which bounds the distinguishing advantage of any statistical test. We use TV rather than max-divergence (D_∞) because the constructions are designed around frequency equalization, which naturally yields TV bounds.

8.1 The Encoding Granularity Principle

Representation uniformity is defined *per cipher map*: the *encoding granularity* is the domain type X, and uniformity applies to the distribution over encodings of individual elements of X. What correlations are hidden depends on what is encoded as a unit. Consider encrypting a pair of Boolean values (a, b) where a and b are correlated.

Component-wise encoding ($p = 1$). Encode a and b as separate cipher values. Each component's marginal distribution is δ-close to uniform, but joint correlations leak.

Pair-wise encoding ($p = 2$). Encode (a, b) as a single cipher value. The distribution over pair encodings is δ-close to uniform; correlations between a and b are hidden. Cost: domain is $\{0, 1\}^2$ (4 elements) rather than $\{0, 1\}$ (2 elements); space per encoding grows.

Whole-state encoding ($p = k$). Encode the entire program state as one cipher value. All correlations are hidden. Cost: domain is the full state space, typically prohibitive.

Definition 8.1 (Entanglement parameter)**.** For a system encoding k correlated Boolean values, the *entanglement parameter* p is the number of values encoded as a single unit. The system has type $\text{cipher}(\{0,1\}^p)^{k/p}$, where each block of p bits is encoded jointly.

Proposition 8.1 (Granularity and privacy)**.** *Let $\hat{f}$ be a cipher map for $f : X_1 \times X_2 \to Y$ with representation uniformity δ. Then the marginal distribution of $\text{enc}((x_1, x_2), k)$ over random $(x_1, x_2) \sim D$ and random k is δ-close to uniform, and the joint distribution of (x_1, x_2) is hidden up to δ.*

*Conversely, let $\hat{f}_1, \hat{f}_2$ be independent cipher maps for $f_1 : X_1 \to Y_1$ and $f_2 : X_2 \to Y_2$, each with representation uniformity δ. Then each marginal cipher value is δ-close to uniform, but correlations between x_1 and x_2 are **not hidden**, even when $\delta = 0$.*

Proof. The first part follows directly from Definition 4.2 applied to the joint cipher map. For the second part: even with $\delta = 0$ (perfect marginal uniformity), enc_i is injective for each fixed k_i. The joint distribution of $(\text{enc}_1(x_1, k_1), \text{enc}_2(x_2, k_2))$ is therefore a bijective image of the joint distribution of $((x_1, k_1), (x_2, k_2))$, preserving all statistical dependence between x_1 and x_2. An adversary observing N draws can estimate this joint distribution and recover the correlation structure of (x_1, x_2). □

Remark 8.1 (Granularity trade-offs in adjacent frameworks)*.* The space-vs-correlation-hiding trade-off via the entanglement parameter p has analogues in oblivious RAM and private information retrieval. ORAM constructions trade access-pattern hiding against per-access overhead (polylogarithmic in storage size); PIR constructions trade query unlinkability against communication and server work (one server or computational PIR pays for unlinkability with cryptographic overhead, multi-server information-theoretic PIR pays with replication). The cipher map granularity knob is the analogous trade-off in the parameterized-leakage setting: hiding the joint distribution of k correlated values requires storage growing as $|Y|^k$, but obviates the per-access protocols ORAM and PIR rely on. Companion work [29] treats the granularity lever in detail within the entropy-ratio framework.

8.2 Compositional Leakage

Even when each cipher map has uniform marginal output, composing multiple cipher maps on a shared input leaks the correlation structure of the latent functions. Consider cipher maps $\hat{f}_1 : \{0,1\}^n \to \{0,1\}^n$ and $\hat{f}_2 : \{0,1\}^n \to \{0,1\}^n$ for latent $f_1 : X \to Y$ and $f_2 : X \to Z$, each with Shannon-optimal acceptance predicates. Individually, the outputs $\hat{y}$ and $\hat{z}$ are uniform. But the untrusted machine observes both $\hat{y} = \hat{f}_1(c)$ and $\hat{z} = \hat{f}_2(c)$ for the same cipher value c. Since $\hat{f}_1$ and $\hat{f}_2$ are deterministic, the pair $(\hat{y}, \hat{z})$ is a deterministic function of c, and the joint distribution of $(\hat{y}, \hat{z})$ preserves the latent correlation between f_1 and f_2 (Proposition 8.1).

If a third cipher map $\hat{f}_3$ takes this correlated pair as input, its output inherits the non-uniformity: $\hat{f}_3$ was constructed assuming a design distribution over its inputs, but the actual input distribution reflects latent correlations. The output $\hat{w}$ is non-uniform even if $\hat{f}_3$'s acceptance predicate is Shannon-optimal for the marginal distribution of W. Every observable correlation between cipher values carries information about the latent data.

Mitigation strategies. Two approaches exist, both costly:

Build the composition directly. Construct a single cipher map for the composed function $g(x) = f_3(f_1(x), f_2(x))$ as one batch construction. The untrusted machine sees only the cipher input $\hat{x}$ and the final cipher output $\hat{w}$; no intermediate values are exposed. This eliminates compositional leakage entirely but costs $O(|X| \cdot (-\log_2 \varepsilon + H(W)))$ space and exponential construction time—the entire program is a single lookup table.

Compose components and inject noise. Keep the practical component-wise evaluation ($\hat{f}_1$, $\hat{f}_2$, $\hat{f}_3$ as separate cipher maps) and add R noise queries for every N real queries. By totality, noise and real queries are indistinguishable to U. The adversary's estimates of pairwise correlations have variance $O(1/N)$; mixing with $R \gg N$ noise pairs dilutes the signal proportionally. This costs bandwidth rather than space.

The fundamental trade-off: constructing the composition as a single cipher map gives full privacy at the cost of space and construction time. Composing component cipher maps is practical but leaks intermediate correlations. Noise injection is an intermediate strategy that trades bandwidth for privacy. The entanglement parameter p controls the granularity of this trade-off.

Honest limitations.

1. Marginal uniformity says nothing about joint distributions.
2. Compositional leakage is inherent in component-wise evaluation; only joint encoding or noise injection can mitigate it.
3. Equality pattern leakage is fundamental: any deterministic encoding leaks the equality pattern of its inputs. Multiple representations ($K(x) > 1$) reduce but do not eliminate this.

9 Discussion

9.1 Relationship to the Bernoulli Model

The correctness parameter η of a cipher map corresponds directly to the error rate in a Bernoulli approximation. A cipher map with correctness η induces a Bernoulli approximation of the latent function f: for each $x \in X$, $\Pr[\text{dec}(\hat{f}(\text{enc}(x, k))) \neq f(x)] = \eta(x)$, where the failing elements are deterministic for a given seed.

The composition bound $\eta_{g \circ f} \leq 1 - (1 - \eta_f)(1 - \eta_g)$ (equality under the re-randomization condition, Definition 7.1) is the standard error accumulation for independent Bernoulli events, identical to the formula derived in the noisy gates framework for AND gates over Bernoulli Booleans. This is not coincidental: a cipher map *is* a Bernoulli approximation, viewed through the lens of hash-based total functions.

The Bernoulli framework organizes approximation into a layered type hierarchy: Bool $\to$ Set $\to$ Map $\to$ Relation $\to$ Type, where each level inherits the error model from the level below. The formalism presented here covers maps ($X \to Y$); relations and algebraic types (sum types, product types) require the extensions developed in the Bernoulli relations and algebraic types work [27]. The BHF (Bernoulli Hash Function) construction from that framework is the optimal cipher map for the batch strategy: it achieves the $-\log_2 \varepsilon + H(Y)$ lower bound with $O(1)$ query time.

9.2 Algebraic Structure

The cipher map abstraction relates to algebraic concepts from the theory of monoid homomorphisms. The decoding function dec is (by construction) a left inverse of enc, and for cipher maps that preserve algebraic structure (e.g., the trapdoor Boolean algebra), dec is a homomorphism from the cipher monoid to the latent monoid—but only up to the equivalence $c_1 \sim c_2 \iff \mathrm{dec}(c_1) = \mathrm{dec}(c_2)$. The quotient $\{0,1\}^n/\sim$ is isomorphic to the latent structure.

We note this connection but do not develop it further. The category-theoretic framing (cipher functors lifting monoids to cipher monoids) is a natural extension but requires careful treatment of the approximate setting: the functor laws hold exactly on the quotient but only approximately on representatives.

9.3 Online Construction

The batch construction developed in §6 requires knowing the domain X and function f at construction time. An alternative *online* strategy encodes sets incrementally via bitwise OR of per-element hashes, with no seed search. Union, intersection, and complement reduce to bitwise OR, AND, and NOT; union is exact but intersection and complement are approximate (cross-element hash collisions and the pigeonhole principle, respectively). This approach—the *trapdoor Boolean algebra*—is developed in a companion paper [27]. The batch and online strategies offer complementary trade-offs: batch achieves information-theoretically optimal space with tunable correctness; online enables incremental construction and algebraic composition at the cost of space exponential in set size.

9.4 Bounded Composition as a Security Feature

A cipher map composition $\mathsf{C}_s(g) \circ \mathsf{C}_s(f)$ stays within the cipher space $\mathsf{C}_s(\cdot)$ indefinitely: there is no construction-imposed limit on chain length under a single secret. But when secrets are rotated to mitigate accumulating leakage, a different picture emerges. The companion cipher rekeying paper [30] introduces a rekeying functor $r_{s_i \to s_{i+1}} : \mathsf{C}_{s_i}(X) \to \mathsf{C}_{s_{i+1}}(X)$, which transports cipher values across secret boundaries while preserving latent meaning. An n-step rekeying chain

$$\mathsf{C}_{s_0}(X) \xrightarrow{r_{s_0 \to s_1}} \mathsf{C}_{s_1}(X) \xrightarrow{r_{s_1 \to s_2}} \cdots \xrightarrow{r_{s_{n-1} \to s_n}} \mathsf{C}_{s_n}(X)$$

admits the bound $H(X \mid \mathcal{V}) \geq H(X) - \log_2(n+1)$ where $\mathcal{V}$ is the adversary's view of all intermediate cipher values [30, Thm. 7.1]: each rekeying step costs at most one bit of latent entropy.

This couples a security guarantee to a complexity restriction. The trusted machine's pre-provisioned set of rekeying maps determines an upper bound on chain length: programs whose composition depth depends on input size cannot be expressed unless the trusted machine refreshes the rekeying budget on demand. In particular, the framework as described here is not Turing-complete: it captures bounded-depth straight-line composition (the typical $n = 2$ to 5 regime of encrypted search, classification cascades, or fixed-depth circuits) but not unbounded recursion or input-dependent loops.

We frame this as a feature rather than a limitation. Sub-Turing computation models have a long history in security contexts—capability machines bound resource use, total functional languages exclude non-termination, structurally recursive type theories exclude unbounded fixpoints—each trading expressivity for static guarantees. Cipher rekeying provides the analogous trade in the information-theoretic setting: chain length n is the explicit expressivity budget, and $\log_2(n+1)$ is the explicit confidentiality cost. An adversary that wants more leakage must pay for it in

chain length, which the trusted machine controls. The expressivity envelope is exactly the kind of computation typical encrypted-search and encrypted-inference workloads need; programs that need more are out of scope by design.

9.5 Open Questions

1. **Max-divergence vs. TV distance for δ.** TV distance bounds distinguishing advantage; max-divergence (D_∞) would give per-element bounds. Which is the right metric depends on the threat model.

2. **Tight bounds for intersection and NOT.** The intersection false positive rate from cross-element collisions in the trapdoor Boolean algebra needs a closed-form expression as a function of $|A|$, $|B|$, $|A \cap B|$, and n. For NOT, deriving $\eta_{\mathrm{NOT}}(|A|, n)$ would complete the parameter instantiation.

3. **Adaptive encoding granularity.** Can the entanglement parameter p be made adaptive—encoding pairs only for correlated values while encoding independent values separately?

4. **Noise-to-signal probability under composition.** Through a chain of m cipher maps, what is the probability that a filler query produces a decodable output at the end? Naively $1 - (1 - \varepsilon)^m$, but this assumes independence of the ε events across maps.

5. **Layered construction laws.** Do the three construction layers (undef, noise, cipher) satisfy formal algebraic laws in the approximate setting? If so, this would enable equational reasoning about cipher map constructions.

6. **Sum-type confidentiality.** Encoding $\mathrm{OT}(X + Y)$ (the sum type as a single opaque value) hides which branch was taken but breaks composability, since downstream cipher maps cannot dispatch on the branch without decoding. Encoding $\mathrm{OT}(X) + \mathrm{OT}(Y)$ (separate opaque values with a visible tag) preserves composability but leaks the branch tag to the untrusted machine. Whether this trade-off is inherent requires formal proof.

9.6 What This Framework Is Not

To set clear expectations:

- *Not ORAM.* ORAM hides access patterns by reshuffling storage with polylogarithmic overhead. Cipher maps use static total functions.

- *Not differential privacy.* Differential privacy bounds what can be learned about a *dataset* from a *query answer*. Cipher maps bound what an untrusted evaluator learns about a *function* from *opaque evaluations*.

- *Not simulation-based security.* We do not claim that the untrusted machine's view can be simulated. The guarantees are parameterized (δ, ε) rather than negligible in a security parameter.

10 Implementation and Evaluation

A reference implementation of the cipher map abstraction is available as the open-source `cipher-maps` Python library.[1] This section connects the formal framework to a concrete encrypted search application on the 20 Newsgroups corpus. Additional empirical investigations (K(x) allocation under finite-sample observation, homophonic prescription on real corpora, online adaptation under distribution drift) appear in companion work [29].

10.1 Reference Implementation

The library provides two construction backends: a brute-force seed search backend (§6.5) for small domains with exact η control, and a perfect hash function (PHF) backend for large domains, built on the `phobic`[2] implementation of the RecSplit minimal perfect hash construction [13]. Cipher Boolean types, arbitrary-codomain cipher maps, and cipher set membership are exposed as composable abstractions with a typed composition discipline that prevents cross-level mixing. The library's confidentiality measurement module computes the entropy ratio e on observed query streams and reports its Fannes–Audenaert lower bound, providing the operational link to Proposition 5.1.

10.2 Application: Encrypted Search

Encrypted search (information retrieval over confidential data by an untrusted provider) is a direct application of the cipher map abstraction. The domain-specific vocabulary maps to the general framework as follows.

Search agent = trusted machine T. Holds the trapdoor s, encodes queries, decodes results.

Encrypted search provider = untrusted machine U. Holds cipher maps and cipher values; evaluates $\hat{f}(c)$ for any $c \in \{0,1\}^n$ without decoding.

Information need = latent function $f : X \to Y$. For keyword search, $f = \mathbf{1}_A$ (set indicator); for ranked retrieval, f is a scoring function.

Hidden query = cipher value $\mathrm{enc}(x, k)$. The provider evaluates the cipher map on this value without learning x.

Secure index = cipher map $\hat{f}$ stored on the provider.

The four properties specialize to encrypted search:

1. **Totality**: the provider can evaluate the index on any bit string; filler queries produce output indistinguishable from real queries.
2. **Representation uniformity**: hidden queries are δ-close to uniform; frequency analysis of query traffic is bounded by δ (Proposition 5.1).
3. **Correctness**: the search agent recovers the correct answer with probability $\geq 1 - \eta$; nonzero η provides plausible deniability (Proposition 10.1 below).
4. **Composability**: Boolean combinations of queries (AND, OR, NOT) compose as cipher map chains with predictable error accumulation (Theorem 7.2).

[1] `https://github.com/queelius/cipher-maps`
[2] `https://pypi.org/project/phobic/`

10.3 20 Newsgroups: Boolean Search Validation

The vocabulary mapping above (§10.2) becomes operational on a real corpus. We instantiate the encrypted search application on the 20 Newsgroups corpus (18,266 documents, 58,903 unique words after tokenization). Each document is encoded as a cipher set of its tokens; queries evaluate Boolean combinations (AND, OR, NOT) over those sets via cipher Boolean types entirely on the untrusted side.

Setup.

- Per-document index: a PHF-backed cipher map (`cipher-maps`, `phobic` backend) over each document's token vocabulary, with values in the cipher Boolean space.
- Cipher Boolean partition: $n = 8$-bit cipher value space, split into True ($p_T = 0.05$), False ($p_F = 0.90$), and noise ($p_N = 0.05$) regions. The partition reflects the empirical sparsity of keyword presence in the corpus (mean per-document presence rate ≈ 0.05 for moderate-frequency terms); matching p_T to the operational rate places the construction in the regime where Property 2 (δ-bounded) is most useful.
- Query distribution: uniform over the corpus vocabulary.
- Hardware: single-thread CPython 3.12, commodity x86_64 desktop. Numbers below are wall-clock from the published library benchmarks in the `examples/` subdirectory; a single run per cell, no cross-replicate aggregation.

Construction. At 5,000 documents, construction completes in 5.9 seconds (843 documents per second). The full 18,266-document index builds in 25.6 seconds. Construction time is dominated by per-document PHF fitting; query latency is a single hash evaluation per cipher map.

Single-term queries. Single-term queries achieve perfect recall (1.0) with precision 0.39. The precision corresponds to expected per-document false positives $5000 \cdot p_T = 250$ over $\approx 4{,}840$ true non-matches, consistent with the $p_T = 0.05$ partition (the "matching p_T" relation is to the per-document false-positive *rate*, not to query precision; precision additionally depends on term frequency).

Multi-term Boolean queries. Multi-term AND queries reduce false positives sharply (from 248 single-term to 12 for 3-term AND) while maintaining perfect recall. The reduction is significant but does not match the naive prediction $5000 \cdot p_T^k = 0.6$ for $k = 3$ that pure independent-FPR multiplication would give. The dominant mechanism is the *noise floor* of cipher Boolean composition: for non-matching documents, each per-term cipher Boolean output is uniformly distributed over the partition, landing in the Noise region with probability $p_N = 0.05$. The AND cipher map, total by Property 1, must produce some output on noise inputs as well; under the construction's design these outputs contribute to the False-positive probability at a rate bounded above by p_T per noise input. The effective k-term FP rate is therefore approximately bounded by

$$\Pr[\mathrm{FP}_{k\text{-AND}}] \lesssim p_T^k + k \cdot p_N \cdot p_T \cdot (1 - p_N)^{k-1}, \tag{11}$$

dominated by the noise term for $k \geq 2$. The observed 3-term FP rate of $12/5000 = 2.4 \times 10^{-3}$ is consistent with this noise-floor regime (the upper bound for $k = 3$ is approximately 6.5×10^{-3});

the precise per-construction FP rate depends on how the cipher Boolean AND routes noise-region inputs through its PHF, characterized in companion work [28].

OR and NOT queries lose recall (0.97 and 0.88 respectively) due to the symmetric mechanism on the recall side: noise inputs to OR hit the False region with probability p_F and to NOT with probability $1 - p_T - p_N$, contaminating disjunctions and complement operations on true matches. The asymmetric recall/precision degradation between AND, OR, and NOT is consistent with the gate-by-gate analysis in §7.1.

Reproducibility. The benchmark scripts and inputs are published with the library at the `examples/` subdirectory; a single command reproduces the table above. The numbers reported here are from one run; variance characterization across replicates and against Bloom-filter and SSE baselines at matched operating points is work in progress (§10.5).

10.4 Deniability via the Correctness Parameter

The 20 Newsgroups validation above used $\eta = 0$ (PHF construction gives exact correctness). But the framework permits tuning $\eta > 0$ to provide explicit deniability, and the relationship between η and posterior confidence is closed-form. The correctness parameter η has a precise Bayesian deniability interpretation for Boolean-valued cipher maps: the posterior on the latent value, given an observed decoded output, can be tuned by the choice of η. This is a theoretical complement to the empirical Boolean search results above; the construction below is a direct restatement of Warner's randomized response mechanism [31] cast in cipher map terms (the correctness parameter η plays the role of Warner's response randomization probability).

Proposition 10.1 (Bayesian deniability)**.** *Let $\hat{f}$ be a cipher map for $f : X \to \{0, 1\}$ with symmetric error rate η (i.e., $\Pr[y = 1 \mid f(x) = 0] = \Pr[y = 0 \mid f(x) = 1] = \eta$), and let $\pi = \Pr[f(x) = 1]$ be the prior. If the adversary observes the decoded output $y = 1$, then*

$$\Pr[f(x) = 1 \mid y = 1] = \frac{\pi(1 - \eta)}{\pi(1 - \eta) + (1 - \pi)\eta}. \tag{12}$$

For $\eta = 0$, the posterior equals 1 (no deniability). For $\eta = 1/2$, the posterior equals π (the observation is useless).

Proof. By Bayes' rule: $\Pr[y = 1 \mid f(x) = 1] = 1 - \eta$ (correct positive) and $\Pr[y = 1 \mid f(x) = 0] = \eta$ (false positive). Applying Bayes' theorem with prior π gives the stated formula. □

10.5 Further Empirical Investigations

The 20 Newsgroups validation and the deniability proposition together exercise the framework on a single workload at a single operating point. The reference implementation supports several empirical investigations across larger scales, alternative workloads, and adversarial settings, all currently in progress:

- Bloom-filter baseline at matched FPR for the 20 Newsgroups workload, relating cipher set space cost to a non-trapdoor baseline at equivalent operating points.
- Construction-time scaling beyond 10^5 documents (the current construction-time table in §6.5 uses toy parameters $m = 100$).

- Empirical entropy ratio e on adversarial query distributions, validating Proposition 5.1 against measured Fannes–Audenaert predictions (the sister work [29] reports related K(x) tuning experiments on this corpus).
- Variance characterization: per-cell replicates, hardware sensitivity, and end-to-end timing under realistic adversary workloads.

Acknowledgments

The constructions in this paper draw on the author's earlier work on the Bernoulli data type library and the trapdoor Boolean algebra.

References

[1] Rakesh Agrawal, Jerry Kiernan, Ramakrishnan Srikant, and Yirong Xu. Order preserving encryption for numeric data. In *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*, pages 563–574, 2004.

[2] Mário S Alvim, Konstantinos Chatzikokolakis, Catuscia Palamidessi, and Geoffrey Smith. Measuring information leakage using generalized gain functions. In *2012 IEEE 25th Computer Security Foundations Symposium*, pages 265–279. IEEE, 2012.

[3] Djamal Belazzougui, Fabiano C Botelho, and Martin Dietzfelbinger. Hash, displace, and compress. In *European Symposium on Algorithms*, pages 682–693. Springer, 2009.

[4] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *Proceedings of the 1st ACM Conference on Computer and Communications Security*, pages 62–73, 1993.

[5] Mihir Bellare, Alexandra Boldyreva, and Adam O'Neill. Deterministic and efficiently searchable encryption. In *Advances in Cryptology–CRYPTO 2007*, pages 535–552. Springer, 2007.

[6] Michael A Bender, Martin Farach-Colton, Rob Johnson, Russell Kraner, Bradley C Kuszmaul, Dzejla Medjedovic, Pablo Montes, Pradeep Shetty, Richard P Spillane, and Erez Zadok. Don't thrash: How to cache your hash on flash. *Proceedings of the VLDB Endowment*, 5(11):1627–1637, 2012.

[7] Burton H Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

[8] Alexandra Boldyreva, Nathan Chenette, Younho Lee, and Adam O'Neill. Order-preserving symmetric encryption. In *Advances in Cryptology–EUROCRYPT 2009*, pages 224–241. Springer, 2009.

[9] David Cash, Stanislaw Jarecki, Charanjit Jutla, Hugo Krawczyk, Marcel-Cătălin Roşu, and Michael Steiner. Highly-scalable searchable symmetric encryption with support for boolean queries. In *Advances in Cryptology–CRYPTO 2013*, pages 353–373, 2013.

[10] David Cash, Paul Grubbs, Jason Perry, and Thomas Ristenpart. Leakage-abuse attacks against searchable encryption. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pages 668–679, 2015.

[11] Reza Curtmola, Juan Garay, Seny Kamara, and Rafail Ostrovsky. Searchable symmetric encryption: Improved definitions and efficient constructions. In *Proceedings of the 13th ACM Conference on Computer and Communications Security*, pages 79–88, 2006.

[12] Yevgeniy Dodis and Adam Smith. Entropic security and the encryption of high entropy messages. In *Theory of Cryptography Conference (TCC 2005)*, pages 556–577. Springer, 2005.

[13] Emmanuel Esposito, Thomas Mueller Graf, and Sebastiano Vigna. RecSplit: Minimal perfect hashing via recursive splitting. In *2020 Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX)*, pages 175–185, 2020.

[14] Bin Fan, Dave G Andersen, Michael Kaminsky, and Michael D Mitzenmacher. Cuckoo filter: Practically better than Bloom. In *Proceedings of the 10th ACM International Conference on emerging Networking Experiments and Technologies*, pages 75–88, 2014.

[15] Michael L Fredman, János Komlós, and Endre Szemerédi. Storing a sparse table with $O(1)$ worst case access time. *Journal of the ACM*, 31(3):538–544, 1984.

[16] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing*, pages 169–178, 2009.

[17] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious RAMs. *Journal of the ACM*, 43(3):431–473, 1996.

[18] Mohammad Saiful Islam, Mehmet Kuzu, and Murat Kantarcioglu. Access pattern disclosure on searchable encryption: Ramification, attack and mitigation. In *Proceedings of the 19th Network and Distributed System Security Symposium*, 2012.

[19] Ari Juels and Thomas Ristenpart. Honey encryption: Security beyond the brute-force bound. In *Advances in Cryptology – EUROCRYPT 2014*, pages 293–310, 2014.

[20] Seny Kamara and Tarik Moataz. Computationally volume-hiding structured encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT 2019)*, pages 183–213. Springer, 2019.

[21] Florian Kerschbaum. Frequency-hiding order-preserving encryption. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pages 656–667, 2015.

[22] Muhammad Naveed, Seny Kamara, and Charles V Wright. Inference attacks on property-preserving encrypted databases. In *Proceedings of the 22nd ACM Conference on Computer and Communications Security*, pages 644–655, 2015.

[23] Claude E Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[24] Gustavus J Simmons. Symmetric and asymmetric encryption. *ACM Computing Surveys*, 11 (4):305–330, 1979.

[25] Geoffrey Smith. On the foundations of quantitative information flow. In *International Conference on Foundations of Software Science and Computational Structures (FoSSaCS 2009)*, pages 288–302. Springer, 2009.

[26] Dawn Xiaodong Song, David Wagner, and Adrian Perrig. Practical techniques for searches on encrypted data. In *Proceedings of the 2000 IEEE Symposium on Security and Privacy*, pages 44–55, 2000.

[27] Alexander Towell. Bernoulli sets and maps: A probabilistic framework for approximate data structures, 2026. Manuscript in preparation. See `https://github.com/queelius/bernoulli_sets`.

[28] Alexander Towell. Algebraic cipher types: A functorial framework for structured computation over trapdoor encodings, 2026. Manuscript in preparation.

[29] Alexander Towell. The entropy ratio: Quantitative confidentiality for trapdoor computing, 2026. Manuscript in preparation.

[30] Alexander Towell. Cipher rekeying via closures, 2026. Manuscript in preparation.

[31] Stanley L Warner. Randomized response: a survey technique for eliminating evasive answer bias. *Journal of the American Statistical Association*, 60(309):63–69, 1965.

[32] Andrew C Yao. Protocols for secure computations. In *23rd Annual Symposium on Foundations of Computer Science*, pages 160–164, 1982.